Software Design Description

Materia: Diseño de Sistemas Escalables

Profesor: David Emmanuel Ramírez Tovar

Alumnos: Federico Humberto Zaragoza Manuel

Ian Eugenio Rodriguez Wong

Jose Juan Diaz Campos

Sergio Isaac Santana Jimenez

Nombre del Proyecto: MySpace

Versión: 1.0

Fecha: 16 de febrero de 2026

1. Introducción

1.1 Propósito

El propósito de este SDD es describir la arquitectura y el diseño del sistema de manejo de condominios MySpace para guiar el desarrollo completo de todo el proyecto, así como definir las tecnologías que se utilizarán a lo largo del todo el proceso.

Este documento está dirigido a los stakeholders del proyecto, quienes se busca aprueben de esta propuesta para satisfacer las necesidades que tienen.

1.2 Alcance

Este sistema está encargado de facilitar el manejo de un condominio. Se tiene contemplado que varios condominios tengan este sistema, por lo que deberá soportar la carga correspondiente. En donde tendremos dos vistas: administración y usuario/inquilino.

La administración será capaz de:

- Revisar el estatus de pago de cada inquilino
- Recibir y revisar tickets de mantenimiento y/o reportes

El inquilino será capaz de:

- Realizar el pago de la renta
- Realizar el pago de servicios (luz, agua, gas, etc.)
- Levantar reportes por fallas o mantenimiento
- Tramitar pases para invitados
- Reservar áreas comunes

La atención para mantenimiento no está contemplada para este sistema, todo este aspecto deberá ser manejado por aparte por la administración, esté sistema solo mantendrá registro de lo reportado por usuarios, más no facilitará el trámite de otro tipo de servicio externo.

2. Visión General del Sistema

2.1 Descripción General

El sistema "MySpace" es una plataforma web diseñada bajo una arquitectura *serverless* enfocada en la alta disponibilidad y la escalabilidad elástica. Actúa como el punto central para la gestión operativa, financiera y de comunicaciones de condominios residenciales.

Dentro de su ecosistema, el sistema funciona como el puente principal entre dos actores clave: la Administración (encargada del monitoreo financiero, gestión de áreas comunes y seguimiento de incidencias) y los Residentes (quienes autogestionan sus pagos, reportes de mantenimiento y visitas).

Para integrarse con el mundo real, el ecosistema de MySpace no operará de manera aislada, sino que interactuará con servicios externos vitales. Dependerá de pasarelas de pago de terceros (como Stripe, PayPal u OpenPay) para procesar de forma segura las transacciones de rentas y servicios. Además, utilizará proveedores de identidad gestionados (Identity Providers como Amazon Cognito) para garantizar un acceso seguro, y servicios de mensajería en la nube (para el envío de notificaciones y pases QR a los residentes y guardias). Su diseño orientado a eventos permite que se adapte de forma automática a los picos de tráfico (por ejemplo, a fin de mes cuando todos los residentes realizan sus pagos) sin degradar la experiencia de los usuarios prioritarios.

2.2 Restricciones de Diseño y Suposiciones

Restricciones:

- **Dependencia del proveedor (Vendor Lock-in):** Para cumplir con los requerimientos de escalabilidad en el tiempo requerido, la arquitectura dependerá fuertemente de los servicios propietarios de AWS (ej. DynamoDB, API Gateway, AWS Lambda), lo que dificulta una futura migración hacia otro proveedor de nube (como Azure o Google Cloud) o a una infraestructura *on-premise*.
- **Presupuesto y Optimización de Costos:** Dado que no se cuenta con un presupuesto amplio para infraestructura base, el diseño está estrictamente limitado a servicios bajo el modelo de "pago por uso" (*pay-as-you-go*), prohibiendo el aprovisionamiento excesivo (over-provisioning) de servidores encendidos 24/7.
- **Tiempo de Desarrollo (Time-to-market):** Al ser un proyecto con fecha límite estricta (académica), el diseño arquitectónico debe priorizar la entrega del Producto Mínimo Viable (MVP) enfocado en pagos, reportes y control de acceso, dejando características secundarias para iteraciones futuras.
- **Separación de Datos por Inquilino (Multi-tenancy):** Por motivos de seguridad y rendimiento, existe la restricción de que los datos financieros y de reportes deben estar lógicamente aislados por cada condominio (Condominio_ID), asegurando una administración más fácil con los condominios identificados y separados.

Suposiciones:

- **Conectividad y Hardware de Usuarios:** Se asume que todos los usuarios (administradores, residentes e invitados) contarán con dispositivos móviles o computadoras con conexión a internet estable y navegadores web modernos para interactuar con el sistema.
- **Disponibilidad de Servicios Externos:** Se asume que las APIs de terceros (pasarelas de pago, servicios de AWS para *serverless*) mantendrán su alta disponibilidad (generalmente 99.9%), ya que una caída en estos servicios paralizaría módulos críticos de MySpace.
- **Gestión de Mantenimiento Física:** Como se acotó en el alcance, se asume que la resolución real de los problemas reportados (ej. plomería, electricidad) será gestionada físicamente por la administración mediante canales externos (llamadas telefónicas, contratación directa); el sistema únicamente actuará como bitácora de seguimiento.

3. Arquitectura del Sistema

3.1 Estilo Arquitectónico

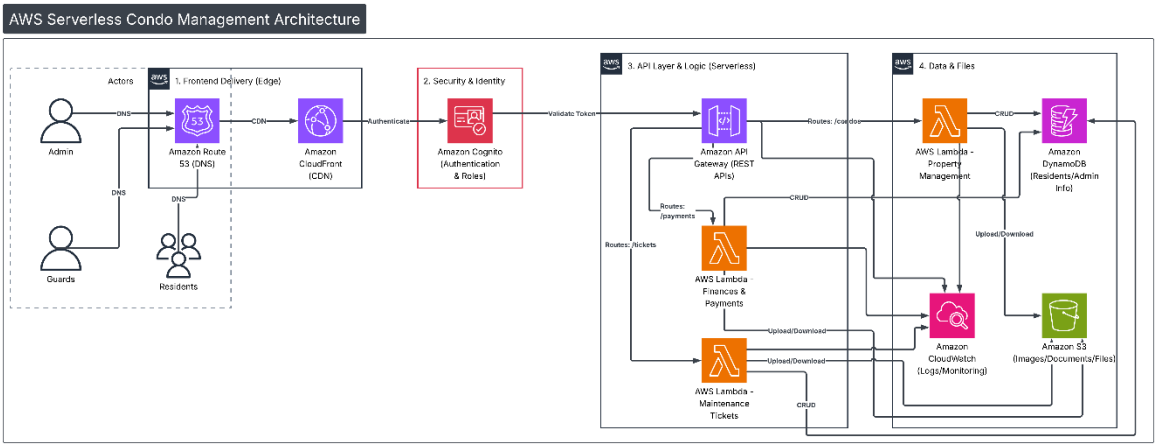
El sistema utilizaría un modelo de Arquitectura Serverless orientada a Microservicios y basada en Eventos, completamente alojada en la nube de AWS. Esta decisión de arquitectura se escoge por la necesidad de soportar cargas de trabajo altamente variables (picos de tráfico a fin de mes) sin incurrir en costos fijos de infraestructura inactiva.

A continuación, se describen los pilares de este estilo arquitectónico implementado:

- **Serverless (Cómputo bajo demanda):** Se delega la administración, el aprovisionamiento y el parcheo de servidores al proveedor de la nube (AWS). Utilizando AWS Lambda, el código de negocio se ejecuta de manera efímera y sin estado (stateless). El sistema escala automáticamente la concurrencia desde cero hasta miles de ejecuciones simultáneas según la demanda en tiempo real, garantizando una alta disponibilidad.
- **Microservicios Desacoplados:** En lugar de construir un backend monolítico, la lógica de negocio se divide en funciones pequeñas e independientes agrupadas por dominio (ej. un microservicio para Pagos, otro para Tickets de Mantenimiento y otro para Control de Accesos QR). Esto permite que, si el módulo de "Tickets" sufre un fallo o saturación, el módulo de "Pagos" siga operando con normalidad (Aislamiento de Fallos).
- **Orientación a Eventos (Event-Driven) vía API:** La comunicación entre el cliente (frontend) y los servicios (backend) se realiza a través de eventos HTTP gestionados por Amazon API Gateway. Cada petición actúa como un disparador (trigger) que invoca a la función Lambda correspondiente de forma asíncrona o síncrona, validando previamente la identidad mediante tokens JWT (Amazon Cognito).

- **Desacoplamiento Frontend/Backend (SPA):** El frontend será una Single Page Application (SPA) utilizando el framework Angular, cuyos archivos estáticos compilados (HTML/CSS/JS) serán alojados en la nube (Amazon S3 + CloudFront). El uso de Angular nos proporciona un buen enrutamiento del lado del cliente y control sobre la lógica y el estado de la aplicación. El frontend no se renderiza en el servidor, se descarga del lado del usuario y consume la API de manera independiente, reduciendo así el ancho de banda y tiempo de latencia.

3.2 Diagrama de Componentes



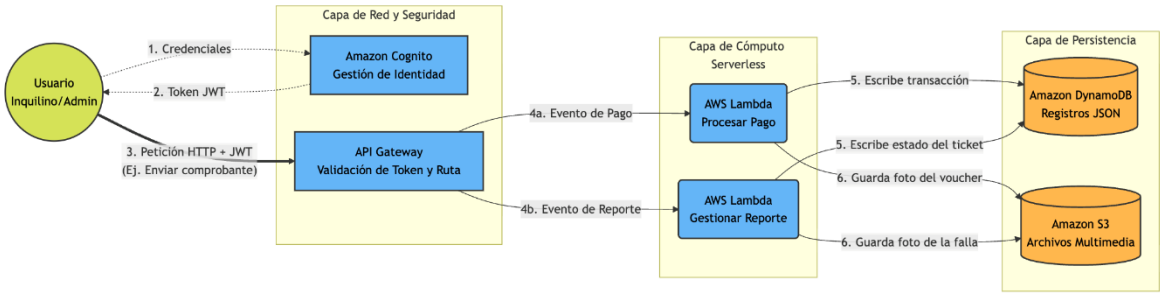
3.3 Stack Tecnológico

- **Frontend:** Angular / Typescript
- **Backend:** Node.js, Express
- **Base de Datos:** Dynamo DB
- **Infraestructura:** Servicios AWS

4. Diseño de Datos

4.1 Diagrama de Flujo de Datos (DFD)

El flujo de datos sigue un modelo orientado a eventos y peticiones HTTP sin estado *(Stateless)*:



4.2 Esquema de Base de Datos

Condominio_Core_Table

Atributo Clave	Tipo	Descripción	Ejemplo de Valor
PK (Partition Key)	String	Identificador principal de agrupamiento.	CONDO#101
SK (Sort Key)	String	Identificador de la entidad específica.	USER#502 o TICKET#99
Data (Atributos)	Map (JSON)	Datos flexibles dependiendo de la entidad.	{"nombre": "Juan", "rol": "Inquilino"}

4.3 Almacenamiento y Caché

- **Almacén Transaccional:** Amazon DynamoDB para todos los registros transaccionales (usuarios, pagos, reportes, reservas).

- **Almacén de Archivos (Object Storage):** Amazon S3 para guardar evidencia de tickets de mantenimiento, comprobantes de pago y códigos QR.

- **Estrategia de Caché:** Se habilitará la caché nativa de Amazon API Gateway para endpoints de lectura frecuente (ej. consultar el listado de amenidades).

5. Diseño de Interfaz

5.1 Interfaz API

1. Módulo CONDOS (Propiedades y Condominios)

- **POST /condominios**
  - **Rol:** Administrador.
  - **Uso:** Registra un nuevo condominio en el sistema.
- **GET /condominios/{id}**
  - **Rol:** Administrador, Residente.
  - **Uso:** Obtiene la información general, amenidades y reglas del condominio.

2. Módulo PAGOS (Finanzas)

- **GET /pagos**
  - **Rol:** Administrador (obtiene lista de todos los residentes con estado "Deudor" o "Al Corriente"), Residente (solo obtiene su propio historial y estado de cuenta).
- **POST /pagos**
  - **Rol:** Residente.
  - **Uso:** Registra un nuevo pago. Si se sube un comprobante físico, devuelve una URL prefirmada de S3 para cargar la imagen.

3. Módulo RESERVAS (Áreas Comunes)

- **GET /reservas**
  - **Rol:** Administrador, Residente.
  - **Uso:** Consulta el calendario de disponibilidad de las amenidades.
- **POST /reservas**

    - **Rol:** Residente.
    - **Uso:** Crea una solicitud de reserva para una fecha, hora y amenidad específica (estado inicial: *Pendiente*).
- **PATCH /reservas/{id}/estatus**
    - **Rol:** Administrador.
    - **Uso:** Actualiza el estado de la reserva a *Aprobada* o *Rechazada*.

4. Módulo TICKETS (Mantenimiento y Reportes)

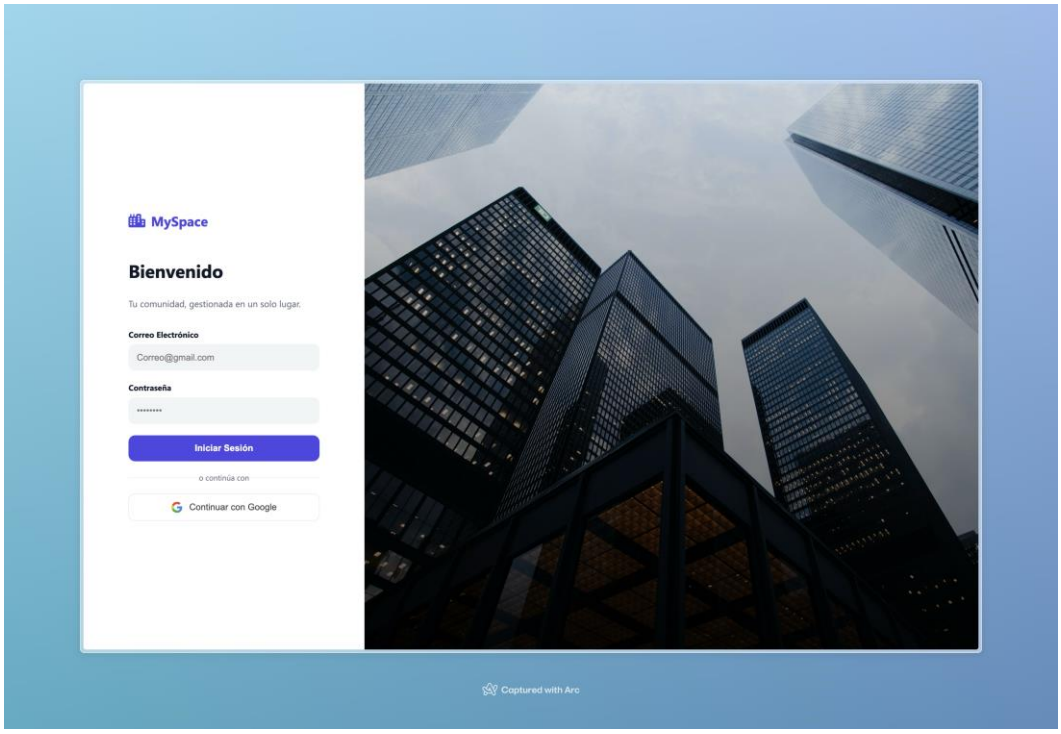
- **GET /tickets**
    - **Rol:** Administrador (ve la bandeja completa de incidencias), Residente (ve únicamente los folios que ha levantado).
- **POST /tickets**
    - **Rol:** Residente.
    - **Uso:** Genera un nuevo reporte de falla.
- **PATCH /tickets/{id}/estatus**
    - **Rol:** Administrador.
    - **Uso:** Cambia el ciclo de vida del ticket (Ej. de *Abierto* a *En Proceso* o *Cerrado*).

5. Módulo VISITAS (Control de Accesos)

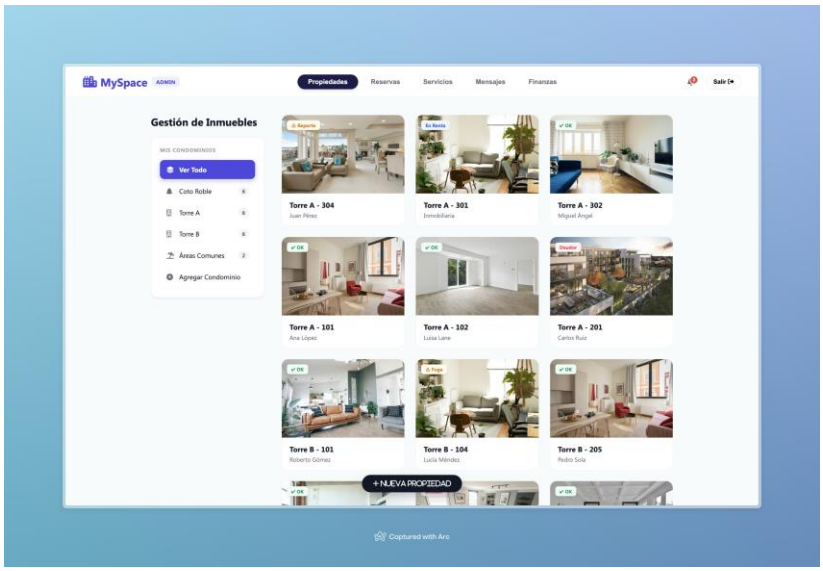
- **POST /visitas**
    - **Rol:** Residente.
    - **Uso:** Registra los datos de un invitado (nombre, fecha) y el sistema genera un token/código QR.
- **GET /visitas/{codigo_qr}/validar**
    - **Rol:** Guardia de Seguridad (Rol sugerido).
    - **Uso:** Escanea el QR. El sistema responde con la validación del pase (Acceso Permitido o Pase Expirado/Inválido) y registra la hora de entrada.

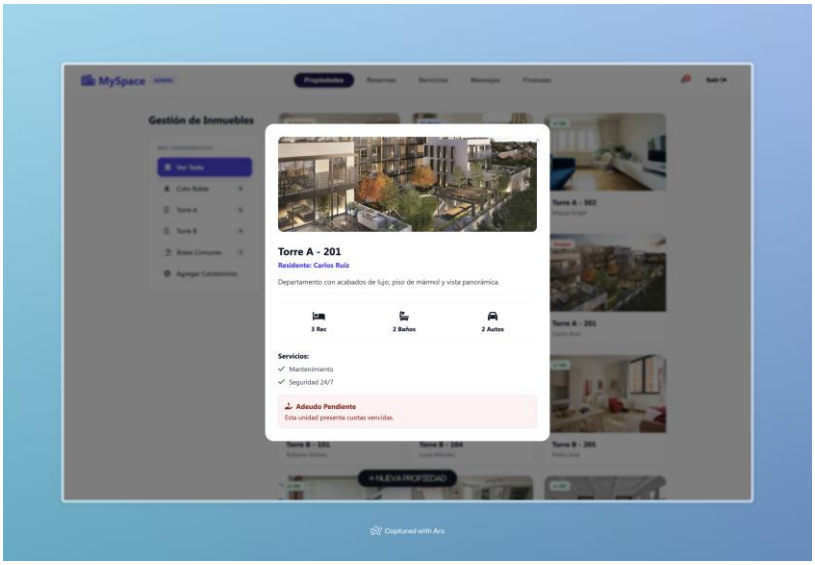
5.2 Interfaz de Usuario (UI)

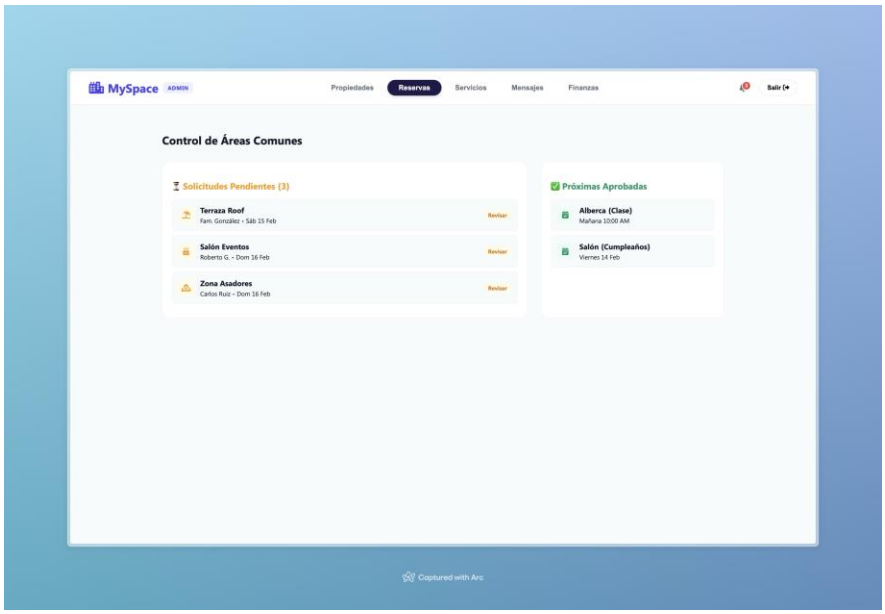
Pantalla de Login

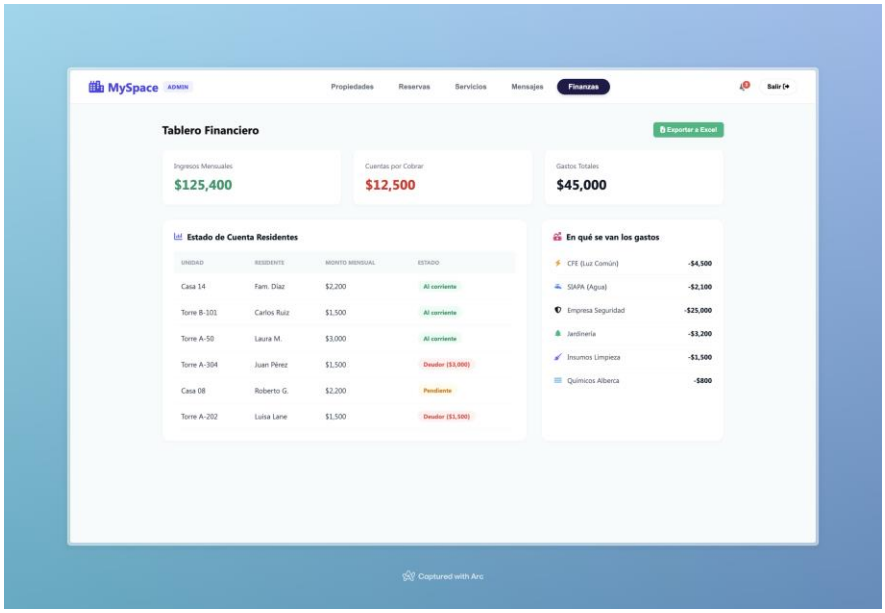


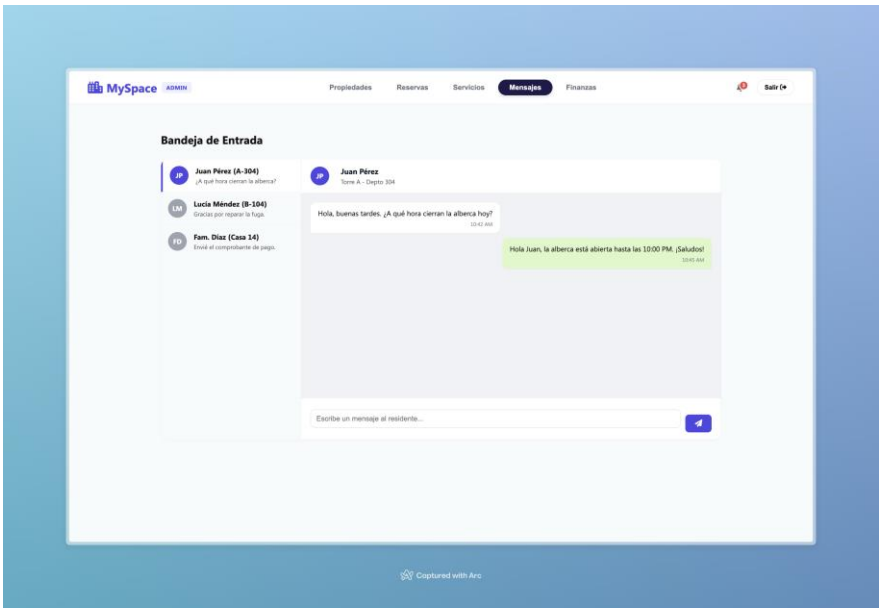
Vista de Administrador



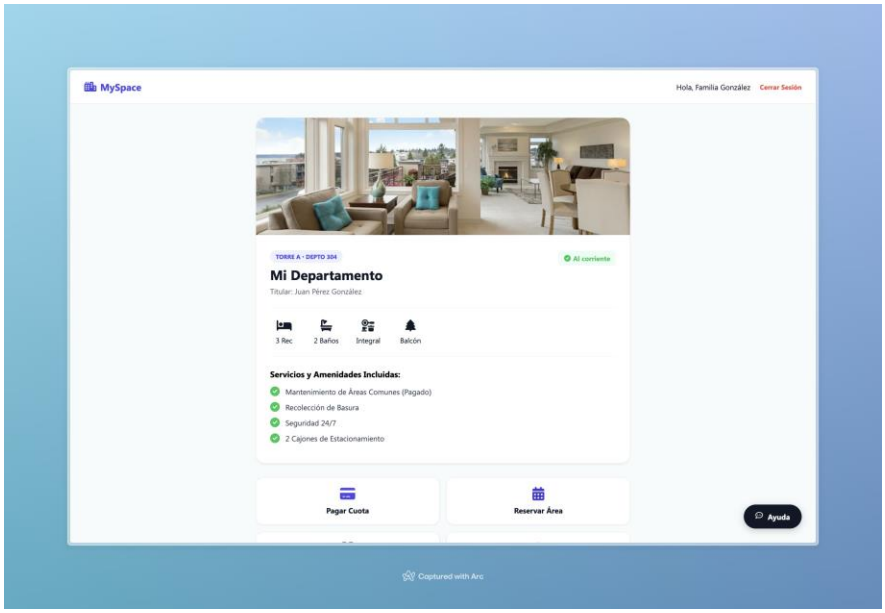


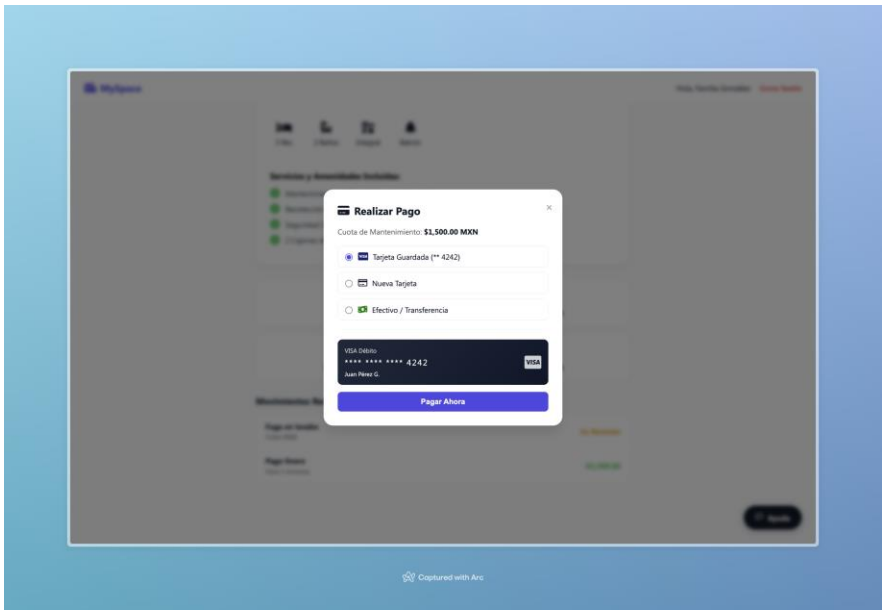

MySpace
ADMIN
Propiedades
Reservas
Servicios
Mensajes
Finanzas
Salir
Control de Áreas Comunes
Solicitudes Pendientes (3)
Terraza Roof
Fam. González · Sáb 15 Feb
Revisar
Salón Eventos
Roberto G. · Dom 16 Feb
Revisar
Zona Asadores
Carlos Ruiz · Dom 16 Feb
Revisar
Próximas Aprobadas
Alberca (Clase)
Mañana 10:00 AM
Salón (Cumpleaños)
Viernes 14 Feb
Captured with Arc

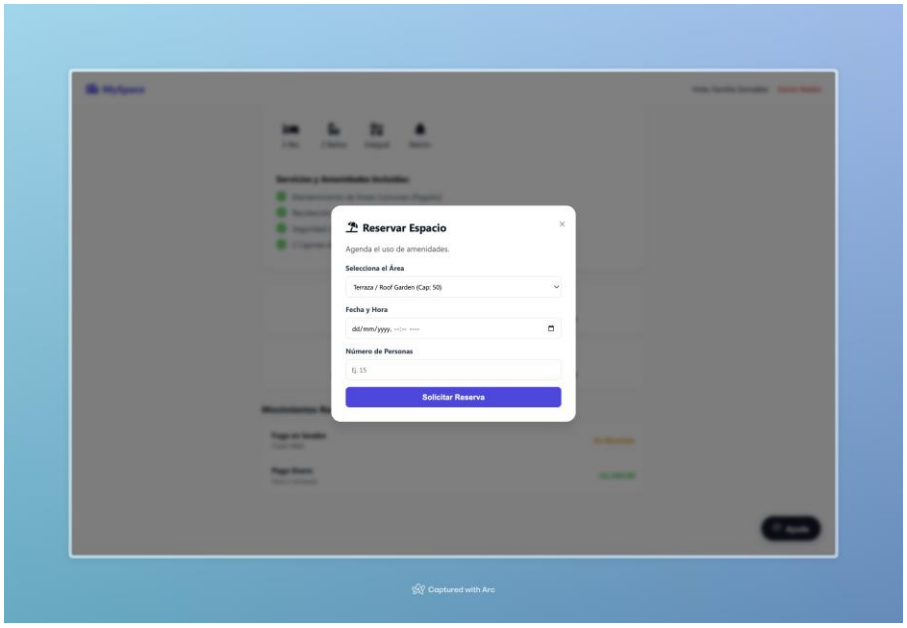

MySpace
ADMIN
Propiedades
Reservas
Servicios
Mensajes
Finanzas
Salir
Tablero Financiero
Exportar a Excel
Ingresos Mensuales
$125,400
Cuentas por Cobrar
$12,500
Gastos Totales
$45,000
Estado de Cuenta Residentes
UNIDAD
RESIDENTE
MONTO MENSUAL
ESTADO
Casa 14
Fam. Díaz
$2,200
Al corriente
Torre B-101
Carlos Ruiz
$1,500
Al corriente
Torre A-50
Laura M.
$3,000
Al corriente
Torre A-304
Juan Pérez
$1,500
Deudor ($3,000)
Casa 08
Roberto G.
$2,200
Pendiente
Torre A-202
Luisa Lane
$1,500
Deudor ($1,500)
En qué se van los gastos
CFE (Luz Común)
-$4,500
SIAPA (Agua)
-$2,100
Empresa Seguridad
-$25,000
Jardinería
-$3,200
Insumos Limpieza
-$1,500
Químicos Alberca
-$800
Captured with Arc



Vista de Residente




Realizar Pago
Cuota de Mantenimiento: $1,500.00 MXN
Tarjeta Guardada (** 4242)
Nueva Tarjeta
Efectivo / Transferencia
VISA Débito
**** **** **** 4242
Juan Pérez G.
VISA
Pagar Ahora
Captured with Arc


Reservar Espacio
Agenda el uso de amenidades.
Selecciona el Área
Terraza / Roof Garden (Cap: 50)
Fecha y Hora
dd/mm/yyyy, --:-- ----
Número de Personas
Ej. 15
Solicitar Reserva
Captured with Arc

6. Requisitos No Funcionales

6.1 Escalabilidad

Al ser una arquitectura Serverless, el sistema no requiere escalar servidores físicos ni virtuales (EC2). En su lugar, el sistema escala de manera nativa y automática por petición:

- **Capa de Cómputo:** AWS Lambda aprovisiona contenedores de ejecución de forma instantánea ante picos de demanda (ej. fin de mes), permitiendo miles de ejecuciones concurrentes y escalando a cero cuando no hay tráfico.
- **Capa de Datos:** DynamoDB se configurará en modo On-Demand (Bajo demanda), adaptando su capacidad de lectura y escritura automáticamente a las cargas de trabajo variables sin intervención manual.

Para controlar los costos y garantizar que el sistema procese todo sin colapsar ni arruinar el presupuesto, se deben establecer límites duros y mecanismos de contención en las siguientes capas de la arquitectura:

1. Amazon API Gateway: Throttling

El primer escudo del sistema es la puerta de entrada de la API. No se debe permitir que las 10,000 peticiones pasen al mismo tiempo hacia la lógica de negocio.

Restricción a aplicar:

Resultado: Los primeros 300 usuarios pasarán. A los 9,700 restantes, el API Gateway les devolverá inmediatamente un error HTTP 429 Too Many Requests. Esto cuesta fracciones de centavo y protege todo lo que está detrás.

2. Frontend en Angular: Manejo del error 429

Para que el usuario no vea un mensaje de "Error en el servidor" cuando el API Gateway rechace su petición por estrangulamiento, el cliente debe ser inteligente.

Restricción a aplicar: Implementar un Retry con Exponential Backoff y Jitter en los interceptores HTTP de Angular.

Resultado: Cuando Angular recibe el error 429, no le dice nada al residente. Oculta el error, espera un tiempo aleatorio (ej. 2 segundos), y vuelve a intentar el pago. Esto "esparce" automáticamente a los 10,000 usuarios a lo largo de un par de minutos sin que ellos se den cuenta de que fueron encolados virtualmente.

3. AWS Lambda: Concurrencia Reservada (Reserved Concurrency)

Por defecto, AWS Lambda puede escalar hasta 1,000 instancias concurrentes en una región. Si las 10,000 peticiones logran pasar, AWS intentará levantar miles de Lambdas, cobrando por cada milisegundo de ejecución en paralelo y saturando la base de datos.

Restricción a aplicar: Configurar la Concurrencia Reservada de la función LambdaPagos a un número bajo y predecible, por ejemplo, 50 instancias máximas.

Resultado: Actúa como un tope de costos de hierro. El sistema jamás ejecutará más de 50 cobros al mismo tiempo, sin importar cuánta demanda haya. Las peticiones excedentes serán estranguladas antes de ejecutar código.

4. Amazon DynamoDB: Capacidad Provisionada vs On-Demand

El modo On-Demand de DynamoDB es excelente para picos impredecibles, pero si 10,000 operaciones de escritura llegan de golpe, el servicio escalará agresivamente y la factura subirá drásticamente por esas unidades de escritura (WCUs).

Restricción a aplicar: Cambiar la tabla Condominio_Core_Table a Capacidad Provisionada y establecer un límite máximo en el Auto Scaling (ej. máximo 500 WCUs).

Resultado: DynamoDB nunca cobrará más del límite establecido. Si el tráfico supera esa capacidad, DynamoDB rechazará las lecturas/escrituras (Throttling interno), y el SDK de AWS dentro de las funciones Lambda se encargará de reintentar la operación automáticamente en milisegundos.

5. La Solución Arquitectónica Definitiva: Amazon SQS (Colas)

Si el requerimiento de negocio exige que los 10,000 usuarios vean una pantalla de "Éxito" instantáneamente sin esperar a que Angular haga reintentos, la arquitectura síncrona debe cambiar.

Restricción a aplicar: Desacoplar el proceso de pago.

Flujo modificado: API Gateway recibe las 10,000 peticiones e inmediatamente las guarda en una cola de Amazon SQS devolviendo un estado HTTP 202 Accepted ("Pago en proceso"). Luego, una sola función Lambda lee de esa

cola en lotes pequeños (ej. 10 pagos a la vez) y va actualizando DynamoDB y llamando a la pasarela a un ritmo constante y barato.

Resultado: Cero estrangulamiento para el usuario, costos de infraestructura completamente planos y controlados, y protección total para la base de datos.

Aplicando la Concurrencia Reservada en Lambda y los Límites de Tasa en API Gateway, el sistema tendrá un techo estricto de gasto, forzando a la arquitectura a procesar la carga masiva mediante tiempo en lugar de mediante fuerza bruta de servidores paralelos.

6.2 Seguridad

- **Autenticación:** La autenticación se manejará mediante JSON Web Tokens (JWT) al momento de iniciar sesión y realizar peticiones.
- **Autorización:** Utilizaremos Amazon Cognito para administrar a los usuarios en base a roles (Administrador, Guardia, Residente) para controlar los accesos y operaciones de cada tipo de usuario.

6.3 Rendimiento

Dado que el sistema opera sobre una arquitectura Serverless, el rendimiento no se mide por la capacidad de un servidor (CPU/RAM), sino por la latencia de respuesta, la concurrencia y la eficiencia del código. Los objetivos de rendimiento se definen de la siguiente manera:

Latencia de la API (Tiempos de Respuesta): El objetivo es que el 95% de las peticiones (Percentil p95) dirigidas a la API (Amazon API Gateway) se resuelvan en menos de 200 milisegundos.

Manejo de Cold Starts (Arranques en frío): Se acepta que el 5% restante de las peticiones (generalmente la primera ejecución después de inactividad) pueda tardar hasta un máximo de 2 segundos en inicializar el contenedor de AWS Lambda. Para las rutas críticas, como el POST /pagos, se evaluará habilitar Provisioned Concurrency (Concurrencia Provisionada) para mantener las funciones "calientes" y garantizar respuestas instantáneas en días de cobro.

Concurrencia y Throughput (Carga de Trabajo): El sistema debe ser capaz de escalar de 0 a 1,000 peticiones por segundo de manera elástica durante los picos de tráfico esperados (los días 15 y 30/31 de cada mes), sin devolver errores de estrangulamiento (Throttling - Error HTTP 429).

Rendimiento de Base de Datos: Amazon DynamoDB deberá mantener una latencia de un solo dígito de milisegundo para todas las operaciones transaccionales de lectura y escritura (GET, PUT, UPDATE) gracias a su diseño basado en particiones.

Mecanismos de Medición:

El cumplimiento de estas métricas se monitoreará en tiempo real utilizando las siguientes herramientas nativas:

AWS CloudWatch: Para medir las métricas Duration (tiempo de ejecución) y ConcurrentExecutions (funciones corriendo al mismo tiempo) de cada Lambda.

AWS X-Ray (Trazabilidad Distribuida): Se instrumentará el código para medir exactamente cuántos milisegundos toma cada salto en la red (ej. de API Gateway → Lambda → DynamoDB) e identificar posibles cuellos de botella.

Pruebas de Carga (Load Testing): Antes del paso a Producción, se simularán escenarios de alto tráfico (picos de fin de mes) utilizando herramientas automatizadas como JMeter o Artillery para validar que los límites elásticos de AWS respondan según lo esperado.

6.4 Fiabilidad y Recuperación

- **Backups:** DynamoDB cuenta con PITR (Point-in-Time Recovery), lo que permite restaurar la base de datos a cualquier segundo en los últimos 35 días para prevenir pérdida de datos por errores humanos.
- **Disponibilidad:** Todos los servicios seleccionados (S3, API Gateway, Lambda, DynamoDB) son Multi-AZ por defecto, lo que significa que AWS replica automáticamente la infraestructura en múltiples zonas de disponibilidad para garantizar un SLA del 99.9%.

7. Diseño Operativo

7.1 Monitoreo y Logging

Dado el ecosistema AWS, el monitoreo se realizará con herramientas nativas *cloud-native*:

- **Amazon CloudWatch:** Para la recolección de logs de todas las funciones Lambda, monitoreo de latencia y configuración de alarmas (ej. si la tasa de errores HTTP 500 supera el 2%).

- **AWS X-Ray:** Herramienta de trazabilidad distribuida (Distributed Tracing) para visualizar el viaje de una petición desde el API Gateway, pasando por Lambda, hasta DynamoDB, e identificar cuellos de botella en el rendimiento.

7.2 Pipeline CI/CD

El proceso de despliegue continuo se orquestará mediante GitHub Actions. Dado que es una infraestructura como código (IaC), se utilizará el framework AWS SAM (Serverless Application Model) o Serverless Framework para empaquetar y desplegar las funciones Lambda y la configuración del API Gateway automáticamente en los ambientes de Staging → Producción.